

Sentinel — Version 2

What to add after the MVP so the product can be hosted, trusted by a team, and used in production.

Version 1 (the MVP) is complete when a developer can add an API, define expected behaviour, monitor it automatically, detect failures, understand what went wrong, and see an in-app alert. Version 2 is the jump from a working personal tool on one machine to a service a team would actually run against production APIs.

Do not start Version 2 by adding dashboards, AI summaries, or billing. Those wait until alerts leave the app, the worker can scale off SQLite, and more than one person can share a project.

1. Goal for Version 2

A small team can invite colleagues, get notified outside the app when an incident opens or resolves, and run Sentinel on a hosted environment with a real database and a background worker that is not tied to a single Next.js process.

2. Priority order

Priority	Theme	Why it matters
P0 — must	Outbound alerts	The MVP loop currently ends in Settings. A page of in-app rows is not an alert.
P0 — must	Production runtime	PostgreSQL, a dedicated worker, Docker that matches how it is hosted.
P0 — must	Teams	Shared projects, roles, and an audit trail of who changed a monitor.
P1 — should	Account safety	Reset, verification, stronger hashing, session revoke — required on the public internet.
P1 — should	Investigation UI	Diffs, latency over time, last-pass vs this-fail, so incidents are diagnosable.
P2 — later	Richer checks	Multi-step flows, OpenAPI/JSON Schema, TLS expiry, maintenance windows, extra regions.
P2 — later	Public API & CI	Create monitors and trigger runs from GitHub Actions after deploy.
V3 — postpone	Commercial extras	Billing, mobile app, AI reports, SEO tooling, WordPress plugin.

Ship P0 before starting P1. P2 is Version 2.1 unless a customer blocks on it.

3. P0 — Outbound alerts

Incidents already open after consecutive failures and resolve on a passing run. Version 2 must deliver that event to a channel the developer already watches.

Requirements

- Notification settings per user and optionally per project or monitor (on / off, channel, destination).
- Email on incident open and incident resolve (transactional provider; no SMTP-in-process as the only path).

- Slack incoming webhook and/or Discord webhook with monitor name, assertion summary, and a link to the incident.
- Generic HTTPS webhook with a signed payload (timestamp + HMAC) so customers can fan-out themselves.
- Retry with backoff; persist delivery status (pending / sent / failed) as the Notification model already allows.
- Do not send a new “open” alert for every failing tick while the same incident remains open.

Done when: a monitor failure produces a Slack or email message without opening Sentinel.

4. P0 — Production runtime

The MVP uses SQLite and an in-process 5-second loop inside Next.js. That is correct for local development. It is not a hosting architecture. The README still describes PostgreSQL; Version 2 should make that true.

Requirements

- PostgreSQL as the production datastore; keep SQLite only as an optional local default.
- Move the scheduler and HTTP worker out of the Next.js web process (second Node service, or Redis + BullMQ / similar).
- Web handlers continue to enqueue only; they must not call customer APIs.
- Docker Compose (or equivalent) that runs app + worker + Postgres, with documented env vars.
- Health endpoints for web and worker; basic metrics (queue depth, run duration, failure rate).
- ENCRYPTION_KEY, session secrets, and database URLs never committed; rotate documented in the README.
- One-command production-shaped local run so hosting is not a surprise.

Done when: two app instances can share one database and only one worker claims each queued run.

5. P0 — Teams

Version 1 is single-user ownership. Version 2 is a project that more than one person can operate without sharing a password.

Requirements

- Organisation or team container above projects (even a thin one: one team per account, then invites).
- Email invite with expiry; accept creates membership.
- Roles: owner, editor, viewer. Viewers cannot change monitors or secrets. Owners can transfer and delete.
- All CRUD remains scoped to membership, not only “created by this user id”.
- Audit log: who created, paused, edited URL/auth, or deleted a monitor.
- No billing in this slice — seats can be unlimited until Version 3.

Done when: two accounts can share a project and only the editor/owner can change a monitor.

6. P1 — Account safety

Required before Sentinel is exposed on the public internet. The MVP session cookie and scrypt hash are enough for a demo, not for strangers.

- Email verification on register; unverified accounts cannot create monitors (or have a tight cap).
- Password reset via time-limited token.
- Change password while logged in; revoke other sessions.
- Session list (device / last used) and sign-out-all.
- Upgrade password hashing to Argon2id (or equivalent) with a migration path from scrypt.
- Optional TOTP second factor for owners.
- Rate limits that survive process restart (Redis) rather than in-memory maps.

7. P1 — Investigation and product polish

Incidents already store expected vs actual. Version 2 should make that the default way people debug, not a raw run list.

- Side-by-side last passing run vs failing run (status, latency, headers, truncated body).
- Latency and pass/fail sparkline on the monitor page.
- Body snippet on failure with redaction of known secret headers.
- Project-level status page (private first; optional public token later).
- Search and filter incidents by monitor, status, and time range.
- CSV/JSON export of runs for a monitor.
- First-run onboarding: create project + example monitor in one guided flow.
- Empty states that tell the user the next action, not a blank table.

8. P2 — Richer checks

- Multi-step monitors: extract a token from step 1 and send it in step 2.
- JSON Schema and/or OpenAPI response assertions.
- TLS certificate expiry check as a first-class assertion or monitor type.
- Maintenance windows that suppress alerting without pausing history.
- Per-monitor timeout/interval already exist; add jitter and timezone-aware schedules.
- Optional second probe region so “down” is not only “this VPS cannot reach it”.

9. P2 — Public API and CI

- Personal or project API tokens (hashed at rest, shown once).
- Documented REST: create/update monitor, enqueue run, fetch latest incident.
- GitHub Action example: after deploy, trigger a run and fail the job on ERROR/FAILED.
- Idempotency keys on enqueue so CI retries do not flood the queue.

10. Explicitly not Version 2

These stay postponed unless a real customer requires them. Mixing them into Version 2 recreates the scope creep the MVP checklist already forbids.

- Billing, plans, usage metering.
- Mobile application.
- AI-generated reports or summaries.

- Google Search Console, rank tracking, backlinks, keyword research.
- WordPress plugin.
- Competitor analysis.
- Multi-tenant white-label / custom domains (beyond a simple public status page).

11. Suggested build sequence

- Slice A — Webhooks + Slack/Discord + email on incident open/resolve.
- Slice B — Postgres migration + worker process + Compose file that a host can run.
- Slice C — Teams, invites, roles, membership-scoped access.
- Slice D — Password reset, email verify, session revoke, Argon2.
- Slice E — Incident diffs, charts, onboarding, export.
- Slice F — Public API tokens + CI example. Richer assertions after that.

Each slice should be usable on its own. Do not block alerts on teams, and do not block teams on charts.

12. Version 2 definition of done

Version 2 is done when all of the following are true:

- A failing production-like API opens an incident and notifies a human on Slack, email, or webhook without anyone watching the dashboard.
- The same incident resolves on recovery and a resolve notification is sent once.
- Two people in one team can operate the same monitors with least-privilege roles.
- The system runs as web + worker + PostgreSQL, and queued runs are claimed atomically.
- A stranger on the internet cannot trivially reset someone else's session or scrape secrets from GET endpoints (true in v1; still true after teams).
- A new user can go from empty account to first green check without reading the README.

If those hold, Version 2 is a product. Everything in P2 is expansion, not the definition.